Molecular Modeling Toolkit Documentation

Release 0.1.0

Molecular Modeling Development Team

Feb 23, 2026

GETTING STARTED

1 Overview **1**
- 1.1 What This Toolkit Can Do . . . 1
- 1.2 Processing Pipeline . . . 1
- 1.3 Voronoi Optimization . . . 2
- 1.4 Substitution System . . . 2
- 1.5 Input and Output . . . 2
- 1.6 GUI . . . 3

2 Installation **5**
- 2.1 Requirements . . . 5
- 2.2 Installation Steps . . . 5

3 How-To Guide **7**
- 3.1 View / Open Crystal Structures . . . 7
- 3.2 Build Supercells . . . 8
- 3.3 Add and Place Molecules . . . 8
- 3.4 Substitute Atoms / Elements . . . 8
- 3.5 Optimize Molecular Orientations . . . 9
- 3.6 Export Structure Files . . . 9
- 3.7 Change Display Settings . . . 9
- 3.8 Select Atoms . . . 10
- 3.9 Undo / Redo . . . 10
- 3.10 TOML Configuration Files . . . 10

4 Quick Start **11**
- 4.1 Running the Pipeline . . . 11
- 4.2 Input Files . . . 11
- 4.3 Output Files . . . 11
- 4.4 Launching the GUI . . . 12
- 4.5 Next Steps . . . 12

5 Tutorial **13**
- 5.1 Step 1: Understanding Molecule Definitions . . . 13
- 5.2 Step 2: Configuring the Input File . . . 14
- 5.3 Step 3: Running the Pipeline . . . 14
- 5.4 Step 4: Checking Output Files . . . 15
- 5.5 Step 5: Reading Voronoi Visualizations . . . 15
- 5.6 Step 6: Interactive GUI Operations . . . 15
- 5.7 Step 7: Substitution Details . . . 16

6 GUI Guide **17**

6.1 Launching . . . 17
6.2 Window Layout . . . 17
6.3 Menu Reference . . . 18
6.4 View Toolbar . . . 20
6.5 Status Bar . . . 21
6.6 Keyboard Shortcuts . . . 22
6.7 Mouse Controls . . . 22
6.8 Dialog Details . . . 23

7 Command-Line Tools **27**
7.1 run.py — Pipeline Execution . . . 27
7.2 molecular_modeling_gui.py — GUI Launch . . . 27
7.3 vis_cif.py — CIF Viewer . . . 28
7.4 edit_and_run.py — GUI Edit + Optimization . . . 28
7.5 Running Tests . . . 28

8 Input / Output File Formats **31**
8.1 File Structure Overview . . . 31
8.2 Input Configuration: input/input.toml . . . 31
8.3 Molecule Definition: data/<name>.toml . . . 34
8.4 Complete Configuration Examples . . . 36

A Voronoi Optimization Details **41**
A.1 Problem Setting . . . 41
A.2 What Are Voronoi Cells? . . . 41
A.3 IoU (Intersection over Union) . . . 41
A.4 Per-Site Optimization Procedure . . . 42
A.5 Dictionary Voronoi Cell Construction . . . 43
A.6 Volume Tiling Verification . . . 43
A.7 Interpreting Optimization Results . . . 44
A.8 Visualization (Voronoi Overlay PNG) . . . 44
A.9 Libraries Used . . . 45

B Architecture **47**
B.1 Pipeline Flow . . . 47
B.2 Module Descriptions . . . 48
B.3 system.info Metadata . . . 49
B.4 GUI Architecture . . . 49

C API Reference **53**
C.1 Module Organization . . . 53
C.2 Main Pipeline . . . 53
C.3 System Construction . . . 54
C.4 Voronoi Geometry . . . 55
C.5 Optimization . . . 60
C.6 Visualization . . . 61
C.7 Output . . . 62
C.8 GUI . . . 63

CHAPTER

ONE

OVERVIEW

Molecular Modeling Toolkit is a molecular modeling toolkit built on ASE (Atomic Simulation Environment). It builds supercells from TOML-based molecular definitions, optimizes molecular orientations via Voronoi cell IoU maximization, and exports to Quantum ESPRESSO / VASP / CIF formats.

1.1 What This Toolkit Can Do

- **Crystal structure construction**: Load CIF files, create supercells, add and place molecules
- **Automatic molecular orientation optimization**: Determine optimal orientations by maximizing Voronoi cell overlap with a reference crystal
- **Atom and molecule substitution**: Bulk element replacement, molecule swapping, individual atom changes (4 substitution methods)
- **Structure file export**: Batch export to Quantum ESPRESSO / VASP / CIF formats
- **Interactive GUI**: 3D visualization, mouse operations, intuitive editing via dialogs

1.2 Processing Pipeline

The core processing consists of a six-stage pipeline:

1. **Build** – Assemble a supercell from TOML configuration and molecule definitions, applying atomic substitutions
2. **Group** – Index atoms by molecular site for subsequent per-molecule operations
3. **Optimize** – Maximize Voronoi cell IoU via Euler angle grid search + Nelder-Mead refinement
4. **Check** – Verify that Voronoi cell volumes sum to the supercell volume
5. **Render** – Generate 3D Voronoi cell overlay visualizations (comparison PNGs before and after optimization)
6. **Export** – Batch export to Quantum ESPRESSO, VASP, and CIF formats

```
input.toml + data/*.toml (molecule definitions)
     |
     v
[1. Build] Supercell generation + atomic substitutions
     |
     v
[2. Group] Per-site atom indexing
     |
     v
[3. Optimize] Voronoi IoU maximization (grid search + Nelder-Mead)
     |
```

(continues on next page)

(continued from previous page)

```
     v
[4. Check] Voronoi volume tiling consistency check
     |
     v
[5. Render] Voronoi overlay PNG generation
     |
     v
[6. Export] qe.in / POSCAR / INCAR / KPOINTS / structure.cif
```

1.3 Voronoi Optimization

The Voronoi optimization, a unique feature of this toolkit, proceeds as follows:

- **Voronoi cell construction**: Cells are computed from molecular center-of-mass coordinates with periodic boundary conditions
- **Volume scaling**: A uniform scale corrects for volume differences between the system and dictionary Voronoi cells
- **Rotation optimization**: A 13x13x13 Euler angle grid search provides a coarse solution, refined by the Nelder-Mead method
- **Evaluation metric**: IoU (Intersection over Union) = intersection volume / union volume (0–1, higher is better)

See *Voronoi Optimization Details* for details.

1.4 Substitution System

Four types of substitution enable flexible crystal structure customization. Substitutions are applied in the following order:

1. **Molecule substitution**: Replace an entire molecule at a specific cell position with a different molecule
2. **Global element substitution**: Replace all atoms of one element across the entire system (e.g., all H to He)
3. **Molecule-atom substitution**: Redefine the atom list of a specific molecule instance (overrides global substitution results)
4. **Individual atom substitution**: Swap a single atom specified by cell, molecule, and atom index

1.5 Input and Output

Input files:

- `input/input.toml` – Lattice vectors, supercell repetition, molecule list, center-of-mass coordinates, substitution rules
- `data/<name>.toml` – Per-molecule atomic positions, lattice vectors, center of mass, element symbols (compact notation `"C6H6"` supported)

Output files:

- `source/structure_out/qe.in` – Quantum ESPRESSO input file
- `source/structure_out/POSCAR`, `INCAR`, `KPOINTS` – VASP input file set
- `source/structure_out/structure.cif` – CIF crystal structure file
- `source/voronoi_pngs/*.png` – Per-site Voronoi cell overlay visualizations

1.6 GUI

A Tkinter-based GUI extending ASE-GUI supports both command-line and interactive workflows. Key features include:

- CIF file loading and 3D visualization
- Interactive molecule addition, deletion, and substitution via dialogs
- Cell parameter editing and supercell construction
- One-click optimization, rendering, and export
- Atom selection, focus display, and measurements (distance, angle, dihedral)
- Per-element and per-molecule-group display, color, and size control

CHAPTER

TWO

INSTALLATION

2.1 Requirements

- Python 3.11 or later (tested with 3.13)
- pip or uv package manager

2.1.1 Dependencies

Key dependencies (managed in `pyproject.toml`):

- **ASE** (3.25.0) — Atomic Simulation Environment
- **NumPy** (2.3.1) — Numerical computing
- **SciPy** (1.16.0) — Voronoi computation, optimization
- **matplotlib** (3.10.3) — Visualization
- **trimesh** (4.7.1) — Mesh operations
- **manifold3d** (3.2.1) — Manifold reconstruction
- **Pillow** (11.3.0) — Image processing

2.2 Installation Steps

2.2.1 1. Clone the repository

```
git clone <repository-url>
cd molecular_modeling
```

2.2.2 2. Create a virtual environment

```
python -m venv .venv
source .venv/bin/activate    # Linux / macOS
.venv\Scripts\activate       # Windows
```

2.2.3 3. Install the package

Using pip:

```
pip install -e .
```

Using uv:

```
uv sync
```

2.2.4 4. Verify installation

Run the test suite to confirm everything is installed correctly:

```
pytest
```

All tests should pass.

2.2.5 Conda environment

You can also set up a Conda environment using `environment.yml`:

```
conda env create -f environment.yml
conda activate molecular_modeling
```

See `CONDA_SETUP.md` for details.

CHAPTER

THREE

HOW-TO GUIDE

A reverse-lookup reference: find what you want to do, then see how to do it. For details, follow the links to the relevant sections.

Categories

- *View / Open Crystal Structures*
- *Build Supercells*
- *Add and Place Molecules*
- *Substitute Atoms / Elements*
- *Optimize Molecular Orientations*
- *Export Structure Files*
- *Change Display Settings*
- *Select Atoms*
- *Undo / Redo*
- *TOML Configuration Files*

3.1 View / Open Crystal Structures

I want to…	How to do it
Open a CIF file and view the structure	GUI: **MoleculeTools > Load CIF…** or from the command line: `python vis_cif.py file.cif` (see *Command-Line Tools* for details)
Open QE / VASP / XYZ or other formats	GUI: **MoleculeTools > Load Structure…** (Ctrl+L). Any format supported by ASE can be read
Load a TOML project	GUI: **MoleculeTools > Load Project…** (Ctrl+T). Select `input.toml` to restore molecule group information
Quickly reopen a recent file	GUI: **Recent Files** menu shows the last 10 files

3.2 Build Supercells

I want to…	How to do it
Repeat a unit cell to create a supercell	GUI: **MoleculeTools > Build Supercell…** and specify repetition counts along [a, b, c]. See *Tutorial* Step 2 for details
View or modify cell parameters (a, b, c, alpha, beta, gamma)	GUI: **MoleculeTools > Cell Parameters…**
Configure via TOML	Set `supercell_repeat = [2, 2, 1]` in `input.toml`. See *Input / Output File Formats* for details

3.3 Add and Place Molecules

I want to…	How to do it
Add a molecule to the supercell	GUI: **MoleculeTools > Add Molecule…** (Ctrl+J). Select from TOML molecule definitions in `data/` folder, specify position and orientation. See *GUI Guide* Add Molecule dialog for details
Move a molecule to a different position	Select atom(s) in the molecule > **Selection > Move Molecule/Atoms…**. Adjust displacement with +/- buttons for each X/Y/Z direction
Rotate a molecule	Select atom(s) in the molecule > **Selection > Rotate Molecule/Atoms…**. Use the trackball or enter Euler angles directly
Drag-move molecules with the mouse	Enable **Selection > Toggle Drag Mode**. Click and drag to reposition directly
Delete a molecule	Select atom(s) in the molecule > **Selection > Delete Molecule…**
Save selected atoms as a new molecule	Select atoms > **MoleculeTools > Save Selection as Molecule…**. A TOML file is created in `data/`

3.4 Substitute Atoms / Elements

I want to…	How to do it
Replace all atoms of one element globally (e.g., all H to D)	GUI: **MoleculeTools > Global Element Substitution…**. Or set `[global_atom_substitutions]` in `input.toml`
Replace an entire molecule with a different one	GUI: **MoleculeTools > Molecule Substitution…**. Orientation is preserved via PCA alignment
Replace a single atom with a different element	GUI: **MoleculeTools > Individual Atom Substitution…**
Apply multiple element substitutions at once (e.g., C to Si and H to F)	GUI: **MoleculeTools > Batch Element Substitution…**. Specify multiple rules in a single dialog
Redefine all atoms in a specific molecule with new elements	GUI: **MoleculeTools > Molecule Atom Substitution…**

3.5 Optimize Molecular Orientations

I want to…	How to do it
Automatically determine molecular orientations via Voronoi optimization	GUI: **Pipeline > Run Optimize (Voronoi)** (Ctrl+P). Maximizes Voronoi cell overlap with the reference crystal. See *Voronoi Optimization Details* for algorithm details
Check optimization results (IoU, rotation angles, scale)	GUI: **Pipeline > Show Optimization Results…** (Ctrl+H). CSV export is also available
Visualize Voronoi cell overlaps	GUI: **Pipeline > Run Render (Voronoi PNGs)…**. Blue = system, Red = dictionary, Orange = intersection
Run the full pipeline at once	GUI: **Pipeline > Run All (Optimize > Render > Export)…**
Run from the command line	`python run.py`. See *Command-Line Tools* for details

3.6 Export Structure Files

I want to…	How to do it
Create a Quantum ESPRESSO input file	GUI: **Export > Quantum ESPRESSO…** or **Export > Export All (QE, VASP, CIF)…** (Ctrl+D)
Create VASP files (POSCAR/INCAR/KPOINTS)	GUI: **Export > VASP…** or **Export > Export All…**
Save as a CIF file	GUI: **Export > CIF…** or **Export > Export All…**
Export all three formats at once	GUI: **Export > Export All (QE, VASP, CIF)…** (Ctrl+D)
Save as a TOML project	GUI: **MoleculeTools > Save Project…** (Ctrl+K). Saves `input.toml` and `data/*.toml`
Save a screenshot of the 3D view	GUI: **View Settings > Save Screenshot…** (Ctrl+Shift+S)

3.7 Change Display Settings

I want to…	How to do it
Show bonds	**View Settings > Show Bonds** (Ctrl+B). Also available from the side panel checkbox
Hide hydrogen atoms	**View Settings > Toggle Hydrogen**
Show/hide specific elements	**View Settings > Element Visibility…**
Color molecules by group	**View Settings > Color by Molecule Group**
Change atom colors and sizes	**View Settings > Atom Appearance…**
Change the background color	Toolbar **BG Color** button (White/Black/Dark/Light/Custom)
Resize the window	**View Settings > Small / Medium / Large / Wide / Custom Size…**
Show/hide the side panel	**View Settings > Toggle Side Panel**

3.8 Select Atoms

I want to…	How to do it
Select all atoms	**Selection > Select All** (Ctrl+A)
Clear selection	**Selection > Deselect All** (Escape)
Select atoms of a specific element (e.g., all C)	**Selection > Select by Element…**
Select a specific molecule group	**Selection > Select by Molecule…**
Select atoms near a point (sphere selection)	Select an atom > **Selection > Select by Sphere…**
Focus on selected molecules only	**Selection > Focus on Selection** (Ctrl+F). Press Ctrl+Shift+F to return to full view
Measure distance between atoms	Select 2 atoms; the distance is shown in the status bar
Measure a bond angle	Select 3 atoms; the angle is shown in the status bar
Measure a dihedral angle	Select 4 atoms; the dihedral angle is shown in the status bar
View measurement history	**MoleculeTools > Measurement Log…**

3.9 Undo / Redo

I want to…	How to do it
Undo the last operation	**MoleculeTools > Undo** (Ctrl+Z). Up to 50 steps
Redo an undone operation	**MoleculeTools > Redo** (Ctrl+Y)

3.10 TOML Configuration Files

I want to…	How to do it
See all input.toml parameters	*Input / Output File Formats* has full field descriptions
Learn how to write molecule definition files (data/*.toml)	See the "Molecule Definition File" section in *Input / Output File Formats*
Learn how to write substitution rules	See the "Substitution Settings" section in *Input / Output File Formats*
See a working example	*Tutorial* has a benzene + naphthalene mixed crystal example

CHAPTER

FOUR

QUICK START

This guide explains how to run the pipeline and check results in the shortest steps.

4.1 Running the Pipeline

With the virtual environment activated, run:

```
cd molecular_modeling
source .venv/bin/activate
python run.py
```

Upon completion, you will see output like:

```
=== Build system from TOML ===
...
=== Optimize Voronoi overlap ===
...
=== Write structure files ===
Written: ./structure_out/qe.in
Written: ./structure_out/POSCAR
Written: ./structure_out/INCAR
Written: ./structure_out/KPOINTS
Written: ./structure_out/structure.cif
```

4.2 Input Files

The pipeline reads the following files:

- `input/input.toml` — Main configuration file (cell vectors, molecule list, repetitions, etc.)
- `data/<name>.toml` — Molecule definition files (atomic positions, lattice vectors, etc.)

4.3 Output Files

The following files are generated in `source/structure_out/`:

File	Description
`qe.in`	Quantum ESPRESSO input file
`POSCAR`	VASP structure file
`INCAR`	VASP calculation settings
`KPOINTS`	VASP k-point mesh
`structure.cif`	CIF structure file

Voronoi cell visualizations are generated in `source/voronoi_pngs/`.

4.4 Launching the GUI

To launch the GUI:

```
python molecular_modeling_gui.py
```

To open an existing CIF file:

```
python vis_cif.py structure.cif
```

The GUI provides:

- Adding and removing molecules
- Element substitutions
- Structure file export
- 3D view manipulation (rotation, translation, zoom)

See *GUI Guide* for details.

4.5 Next Steps

- *Tutorial* — Detailed tutorial with benzene + naphthalene example
- *GUI Guide* — Full GUI operations guide
- *Command-Line Tools* — Command-line tools usage
- *Input / Output File Formats* — Complete TOML configuration reference
- *Architecture* — Internal architecture explanation

CHAPTER

FIVE

TUTORIAL

This tutorial walks through all pipeline steps using a benzene-naphthalene mixed crystal as an example.

5.1 Step 1: Understanding Molecule Definitions

Let's examine the contents of `data/benzene.toml`:

```toml
molecule_name = "benzene"

symbols = [
  "C", "C", "C", "C", "C", "C",
  "H", "H", "H", "H", "H", "H",
]

use_fractional = false

positions = [
  [ -1.113710, -0.097913, -0.811833 ],
  [  1.113710,  0.097913,  0.811833 ],
  # ... (coordinates for all 12 atoms)
]

lattice_vectors = [
  [ 5.522000,  0.000000,  0.000000],
  [ 0.000000,  5.439600,  0.000000],
  [-2.693272,  0.000000,  7.184363],
]

num_molecules = 2

center_of_mass = [
  [ 0.000000,  0.000000,  0.000000],
  [-1.346636,  2.719800,  3.592182],
]
```

Field descriptions:

- `molecule_name`: Molecule identifier (referenced by `molecules` in `input.toml`)
- `symbols`: List of element symbols. Compact notation like `"C6H6"` is also supported
- `use_fractional`: If `true`, coordinates are interpreted as fractional
- `positions`: Atomic Cartesian coordinates (Angstrom). Fractional if `use_fractional = true`

- `lattice_vectors`: Lattice vectors (rows define a, b, c)
- `num_molecules`: Number of molecules per unit cell
- `center_of_mass`: Center of mass position for each molecule

See *Input / Output File Formats* for full key descriptions.

Naphthalene (`data/naphthalene.toml`) follows the same structure, using compact notation `"C10H8"` for element symbols.

5.2 Step 2: Configuring the Input File

`input/input.toml` specifies how to build the supercell:

```
# Random seed
rng = 1

# Lattice vectors (Angstrom)
cell = [
  [ 5.522000,  0.000000,  0.000000 ],
  [ 0.000000,  5.439600,  0.000000 ],
  [-3.501254,  0.000000,  9.339672 ],
]

# Supercell repetition counts [a, b, c directions]
supercell_repeat = [1, 1, 2]

# Molecule names to use
molecules = ["benzene", "naphthalene"]

# Whether to use fractional coordinates
use_fractional = false

# Center of mass for each molecule
center_of_mass = [
  [  0.000000,   0.000000,   0.000000 ],
  [ -1.750627,   2.719800,   4.669836 ],
]
```

`supercell_repeat = [1, 1, 2]` creates a supercell doubled along the c-axis. Two molecule types (benzene and naphthalene) are placed at their respective center-of-mass positions.

5.3 Step 3: Running the Pipeline

```
python run.py
```

The pipeline processes in the following order:

1. **Build**: Read TOML files and construct ASE `Atoms` object
2. **Group**: Group atoms by `mol_id`
3. **Optimize**: Maximize Voronoi IoU via Euler angle grid search
4. **Check**: Verify Voronoi volume tiling consistency
5. **Render**: Generate 3D Voronoi cell overlay visualizations

6. **Export**: Write QE, VASP, CIF files

5.4 Step 4: Checking Output Files

5.4.1 Structure Files

The following files are generated in `source/structure_out/`:

qe.in (Quantum ESPRESSO):

```
&CONTROL
  calculation = 'scf'
  prefix = 'molecule'
  outdir = './tmp'
/
&SYSTEM
  ecutwfc = 40
  ecutrho = 320
/
...
```

POSCAR (VASP): Atomic positions in fractional coordinates.

structure.cif: Output in universal CIF format.

5.5 Step 5: Reading Voronoi Visualizations

PNG images in `source/voronoi_pngs/` show the Voronoi cell overlay for each site:

- **Blue translucent polyhedron**: System Voronoi cell (actual cell partition)
- **Red translucent polyhedron**: Dictionary Voronoi cell (optimization target)
- Higher IoU (Intersection over Union) values indicate better molecular orientation reproduction

5.6 Step 6: Interactive GUI Operations

See *GUI Guide* for the full GUI reference.

5.6.1 Loading a CIF file

```
python vis_cif.py source/structure_out/structure.cif
```

Or from the GUI menu:

MoleculeTools → Load CIF…

5.6.2 Adding Molecules

1. Select **MoleculeTools → Add Molecule…**
2. Choose a molecule from the dropdown (e.g., benzene)
3. Enter placement position coordinates
4. Click **OK**

5.6.3 Substitution Operations

The GUI supports 4 types of substitutions:

1. **Global Element Substitution**: Replace all atoms of a specified element
2. **Molecule Substitution**: Replace a molecule at a specific site
3. **Molecule Atom Substitution**: Change the atom list within a molecule
4. **Individual Atom Substitution**: Replace a single atom

5.6.4 Export

Export → Export All (QE, VASP, CIF)… exports to all formats at once.

5.7 Step 7: Substitution Details

`input.toml` supports 4 types of substitution tables.

5.7.1 Molecule Substitutions

Replace a molecule at a specific cell position with a different molecule:

```
[[molecule_substitutions]]
cell_index       = [0, 0, 1]    # Cell position in supercell
molecule_index   = 1            # Molecule index (0-based)
new_molecule     = "benzene"    # Replacement molecule name
```

5.7.2 Global Atom Substitutions

Replace all atoms of a specified element throughout the system:

```
[[global_atom_substitutions]]
old_symbol = "H"     # Original element
new_symbol = "He"    # Replacement element
```

5.7.3 Molecule Atom Substitutions

Replace the entire atom list of a specific molecule instance:

```
[[molecule_atom_substitutions]]
cell_index     = [0, 0, 1]
molecule_index = 1
new_symbols    = ["Si6O6"]    # Compact notation supported
```

5.7.4 Individual Atom Substitutions

Replace a single atom:

```
[[atom_substitutions]]
cell_index     = [0, 0, 0]
molecule_index = 1
atom_index     = 2           # Atom index within molecule (0-based)
new_symbol     = "Fe"
```

CHAPTER

SIX

GUI GUIDE

The Molecular Modeling GUI is a graphical interface extending ASE-GUI. It provides interactive operations for adding molecules, substitutions, and exporting structure files.

6.1 Launching

```
# Launch with empty structure
python molecular_modeling_gui.py

# Launch with an existing structure file
python molecular_modeling_gui.py structure.cif

# Launch as CIF viewer
python vis_cif.py structure.cif

# Launch GUI from pipeline (edit_and_run.py)
python edit_and_run.py
```

6.2 Window Layout

The GUI window consists of:

1. **File panel (top)**: Icon button groups (described below)
2. **3D viewport**: Molecular structure display (rotatable, translatable, zoomable) with XYZ axis indicator in bottom-left
3. **Toolbar**: Translate / Rotate / Zoom buttons
4. **Bottom panel**: Molecule Tools / Structure Tools icon button groups
5. **Side panel (right)**: View settings, Element Controls, Molecule Controls (collapsible)
6. **Status bar**: Information about selected atoms

6.2.1 Changing Window Size

Use the **View Settings** menu to change the window size:

- **Small (450x450)**: Compact display
- **Medium (700x700)**: Standard size (default)
- **Large (1000x800)**: Large display
- **Wide (1200x600)**: Widescreen display

- **Custom Size…**: Specify any size

You can also resize the window by dragging its borders.

6.2.2 File Panel (top)

Icon buttons at the top of the window provide quick access to common operations.

Group	Buttons
Input	Load Structure / Load Project (TOML project)
Output	Save Project / Export Structure / Save Selection as Molecule / Screenshot
Edit	Undo / Redo
Pipeline	Run Optimize / Run All / Show Results / View PNGs

6.2.3 Bottom Panel

Icon button groups below the 3D viewport.

- **Molecule Tools**: Select / Move / Rotate / Delete / Drag (drag mode)
- **Structure Tools**: Add Molecule / Substitution / Supercell / Cell Parameters

6.3 Menu Reference

6.3.1 MoleculeTools Menu

Menu Item	Description
Undo (Ctrl+Z)	Undo the last operation (up to 50 steps)
Redo (Ctrl+Y)	Redo the last undone operation
Load Structure… (Ctrl+L)	Load a structure file (CIF, XYZ, POSCAR, QE .in, PDB, GEN, etc.)
Load CIF…	Load a CIF file and display the structure
Add Molecule… (Ctrl+J)	Add a registered molecule to the system
List Molecules/Groups	List all molecules and groups in the system
Save Selection as Molecule…	Save selected atoms as a new molecule TOML file
Global Element Substitution…	Replace all atoms of a specified element
Molecule Substitution…	Replace a molecule at a specific site
Molecule Atom Substitution…	Replace the atom list within a molecule
Individual Atom Substitution…	Replace a single atom
Batch Element Substitution…	Define and apply multiple element substitution rules at once (e.g., C→Si, H→F)
Build Supercell…	Build a supercell with specified repeat factors
Cell Parameters…	Display and edit lattice parameters (a, b, c, α, β, γ). Also shows lattice vectors and volume. Apply rescales atomic positions
Save Project… (Ctrl+K)	Save project files (input.toml + data/*.toml) to a chosen directory
Load Project… (Ctrl+T)	Load a project from a directory and build the structure
Measurement Log…	Show measurement log (distance/angle/dihedral history)

6.3.2 Recent Files Menu

Displays up to 10 recently opened CIF files. Click a filename to open it directly. History is saved in `~/.molecular_modeling_recent.json`.

6.3.3 Export Menu

Menu Item	Description
Export All (QE, VASP, CIF)… (Ctrl+D)	Export to all formats at once
Quantum ESPRESSO…	Export as QE input file (.in)
VASP…	Export as POSCAR file
CIF…	Export as CIF file

6.3.4 Selection Menu

Menu Item	Description
Select All (Ctrl+A)	Select all visible atoms
Deselect All (Escape)	Clear all selections
Select by Element…	Select atoms by element with real-time preview
Select by Molecule…	Select atoms by molecule group with color swatches and preview
Select by Sphere…	Select atoms within a radius with slider-based real-time preview
Delete Selected Atoms (Delete)	Delete selected atoms (with undo support)
Delete Molecule…	Delete the entire molecule group of the selected atoms
Move Molecule/Atoms…	Translate selected molecule/atoms by a specified vector
Rotate Molecule/Atoms…	Rotate selected molecule/atoms with trackball widget
Focus on Selection (Ctrl+F)	Open the Focus dialog with 3D preview
Exit Focus Mode (Ctrl+Shift+F)	Exit focus mode and restore all atom visibility
Toggle Drag Mode	Toggle drag mode for direct atom dragging

6.3.5 Pipeline Menu

Menu Item	Description
Run Optimize (Voronoi) (Ctrl+P)	Run Voronoi IoU optimization on the current structure. Results (IoU values, Euler angles) are displayed in a dialog
Run Render (Voronoi PNGs)…	Generate 3D Voronoi cell overlay PNG visualizations
Run Export (QE, VASP, CIF)…	Export structure files to a selected directory
Run All (Optimize → Render → Export)…	Run the full pipeline: optimize → render → export
Show Optimization Results… (Ctrl+H)	Display optimization results (IoU values, Euler angles, scales) in a table. Supports CSV export and Voronoi PNG browsing

Note: Pipeline operations require a structure with `mol_id` information (i.e., built from TOML). Processing runs in a background thread so the GUI remains responsive.

6.3.6 View Settings Menu

Menu Item	Description
Atom Appearance…	Open the atom appearance panel to customize colors, color modes (by element/molecule group/tag), and atom sizes
Color by Molecule Group	Color atoms by `mol_id` using distinct colors per molecule group
Reset Colors	Reset colors to ASE default (jmol)
Show Bonds (Ctrl+B)	Toggle display of bonds (lines between atoms). Bonds are auto-detected based on covalent radii
Toggle Hydrogen	Toggle visibility of hydrogen atoms
Element Visibility…	Open dialog to show/hide atoms by element
Toggle Side Panel	Show or hide the side panel (quick-access view controls)
Save Screenshot… (Ctrl+Shift+S)	Save the current 3D view as a PNG image
Small / Medium / Large / Wide	Window size presets
Custom Size…	Specify any window size

6.3.7 Help Menu

Menu Item	Description
Keyboard Shortcuts…	Show a dialog listing keyboard shortcuts and mouse controls
Open User Guide…	Open the built HTML documentation in the browser (requires `docs/_build/html/`; shows build instructions if not available)
About…	Show version information

6.4 View Toolbar

The toolbar below the 3D viewport provides view controls. Each button supports press-and-hold for continuous operation.

Section	Buttons	Description
Translate	◄ ► ▲ ▼	Move the view up/down/left/right
Rotate	X- X+ Y- Y+ Z- Z+	Rotate ±5° around X/Y/Z axis
Zoom	- +	Zoom in/out
BG Color	White / Black / Dark / Light / Custom	Change background color preset or pick a custom color
Actions	↩ ↪ ✕ ✦ ↻	Undo / Redo / Delete selected / Move / Rotate (each button has a tooltip)
Drag	[Drag]	Toggle drag mode: click to activate, then drag selected atoms to reposition them

6.4.1 Tooltips

All toolbar buttons display tooltips (floating help on mouse hover). Hover the mouse cursor over any button to see a yellow popup with the operation description and keyboard shortcuts.

6.4.2 Side Panel

A side panel is displayed on the right side of the window for quick access to common view settings. The panel is scrollable — use the **mouse wheel** to scroll up/down when content overflows.

- **Focus / Unfocus**: Buttons to enter/exit focus mode
- **Color Mode**: Switch color mode via radio buttons (By Element / By Molecule / By Tag)
- **Show Bonds**: Checkbox to toggle bond display (Ctrl+B)
- **3D Spheres**: Enable/disable 3D shaded sphere rendering
- **Element Controls** *(collapsible)*: Click the header to expand/collapse. Per-element row with [visibility checkbox][color swatch][size slider]. Global atom size scale slider (0.3–3.0) included
- **Molecule Controls** *(collapsible)*: Click the header to expand/collapse. Per-molecule-group row with [visibility checkbox][color swatch][select button]. **In focus mode**, checkboxes act as focus add/remove controls
- **Atom List** *(collapsible, collapsed by default)*: Click the header to expand. Shows a table of all atoms with index (#), element, X/Y/Z coordinates, and molecule group ID. Click a row to select that atom in the 3D view
- **Status info**: Real-time display of atom count, selection count, element breakdown, group count

The side panel can be shown/hidden via **View Settings → Toggle Side Panel**.

6.5 Status Bar

The status bar at the bottom shows context-dependent information:

- **No atoms selected**: System overview — atom count, elements, molecule groups, cell size (e.g., `48 atoms | (C, H) | 4 groups | Cell: 5.5x5.4x18.7 Å`)
- **1 atom selected**: Atom details — index, element name, coordinates, molecule name (e.g., `#3 Carbon (C): 1.114, 0.098, 0.812 Å | mol_id=0 (benzene)`)
- **2 atoms selected**: Inter-atomic distance (MIC corrected) with molecule name (e.g., `#0(C)–#6(H): 1.089 Å | benzene`)
- **3 atoms selected**: Bond angle (MIC corrected) with molecule name (e.g., `#0(C)–#1(C)–#2(C): 120.0° | benzene`)
- **4 atoms selected**: Dihedral angle (MIC corrected)

Measurements are automatically logged and can be reviewed via **MoleculeTools → Measurement Log…**.

6.6 Keyboard Shortcuts

Shortcut	Action
Ctrl+Z	Undo
Ctrl+Y	Redo
Ctrl+A	Select all visible atoms
Escape	Deselect all
Delete / Backspace	Delete selected atoms
Ctrl+L	Load structure file (CIF, XYZ, POSCAR, etc.)
Ctrl+J	Add molecule
Ctrl+D	Export to all formats
Ctrl+P	Run Voronoi IoU optimization
Ctrl+H	Show optimization results
Ctrl+K	Save project
Ctrl+T	Load project
Ctrl+B	Toggle bond display
Ctrl+F	Focus on selection (show only selected atoms)
Ctrl+Shift+F	Exit focus mode (restore all atoms)
Ctrl+Shift+S	Save screenshot

Standard ASE-GUI keyboard shortcuts (Ctrl+O to open, Ctrl+S to save, etc.) are also available.

6.7 Mouse Controls

Standard ASE-GUI mouse controls are available, plus a right-click context menu:

- **Left click**: Select an atom
- **Shift + left click**: Add to selection
- **Ctrl + left click**: Toggle selection
- **Right-click**: Context menu (Select Molecule Group / Move / Rotate / Delete)
- **Left drag**: Rotate view
- **Right drag** (or Shift + right drag): Translate view
- **Mouse wheel**: Zoom

6.7.1 Axis Indicator

An XYZ axis indicator is displayed in the bottom-left corner of the 3D viewport. It updates in real-time with the current view rotation so you can always see which direction you're looking. Axes are color-coded: **X** = red, **Y** = green, **Z** = blue.

6.7.2 Right-Click Context Menu

Right-clicking in the 3D viewport shows a context menu with quick access to:

- **Select Molecule Group**: Select all atoms in the molecule group of the clicked atom
- **Move…**: Open move dialog
- **Rotate…**: Open rotate dialog
- **Delete Selected**: Delete currently selected atoms
- **Delete Molecule**: Delete the molecule of the selected atom

6.7.3 Screenshot

Use **Ctrl+Shift+S** or the camera icon in the file panel to save the current 3D view as a PNG file. If Pillow is installed, the screenshot is saved as PNG; otherwise as EPS.

6.8 Dialog Details

6.8.1 Focus Selection Dialog

When you run **Focus on Selection** (Ctrl+F) with atoms selected, a preview window appears.

The window is **non-modal**, so clicking atoms in the main view is possible.

- **3D Preview** (top): Focused atoms are shown in color; non-focused atoms are shown as faded gray. The plot can be rotated and zoomed
- **Status**: Shows the number of focused atoms
- **Unit**:
  - **Atom**: Add/delete individual atoms
  - **Molecule**: Add/delete the whole molecule group of the clicked atom
- **Molecules**: Each molecule group is listed with a checkbox and color swatch. Toggling checkboxes adds/removes groups from the focus set

Auto-toggle behavior:

The main view highlights currently focused atoms as selected (highlighted). No Add/Delete mode switching is needed — clicking automatically toggles:

- Click an **unfocused** atom → **adds** it to the focus set
- Click a **focused** atom → **removes** it from the focus set

Regular click, Ctrl+click, and Shift+click all work.

Example workflow:

1. Select atoms and click Focus → preview shows molecules of selected atoms, main view highlights the focus set
2. Set Unit to Molecule, click another atom → its molecule is added to the preview, main view highlight updates
3. Set Unit to Atom, click a specific focused atom → only that atom is removed
4. Use the Molecules checkboxes in the dialog to add/remove whole molecule groups
5. Click OK to apply focus mode, or Cancel to restore the original state

Side panel controls during focus mode:

While in focus mode, the Molecule Controls checkboxes in the side panel act as focus add/remove controls:

- Check a molecule → adds it to the focused view
- Uncheck a molecule → removes it from the focused view

6.8.2 Add Molecule Dialog

- **Molecule**: Select from molecules defined in `data/*.toml`
- **Position (X, Y, Z)**: Enter placement position in Cartesian coordinates
- **Add Mode**:
  - **Single**: Add one molecule at the specified position
  - **Unit Cell**: Add all molecules from the molecule definition at once (also sets lattice vectors)

- **3D Preview**: Preview of the selected molecule

6.8.3 Save Selection as Molecule Dialog

1. Select atoms in the 3D viewport (Shift + click for multiple selection)
2. Choose **MoleculeTools → Save Selection as Molecule…**
3. Enter molecule name and number of molecules per unit cell
4. Click **OK** to save as `data/<name>.toml`

6.8.4 Global Element Substitution Dialog

- **Old Symbol**: Element to replace (selected from existing elements)
- **New Symbol**: Replacement element symbol
- All matching atoms are replaced at once

6.8.5 Molecule Substitution Dialog

- **Molecule Index**: Index of the molecule to replace (auto-filled if atoms are selected)
- **New Molecule**: Select replacement molecule from dropdown

6.8.6 Molecule Atom Substitution Dialog

Replace all atoms of a specific molecule with a new element list.

- **Molecule Index**: Target molecule index (auto-filled when atoms are selected)
- **New Symbols**: New element symbol list. Compact notation (e.g., `Si606`) supported
- The atom count stays the same; only element symbols are changed

6.8.7 Individual Atom Substitution Dialog

Replace a single atom within a molecule with a different element.

- **Molecule Index**: Target molecule index (auto-filled when atoms are selected)
- **Atom Index**: Atom index within the molecule (0-based)
- **New Symbol**: Replacement element symbol

6.8.8 Batch Element Substitution Dialog

Define and apply multiple element substitution rules at once.

- Each row specifies "old element → new element"
- **+** button to add rules, **x** button to remove
- Example: C→Si and H→F can be applied simultaneously
- **OK** applies all rules at once

6.8.9 Build Supercell Dialog

Build a supercell by repeating the current structure.

- **a, b, c**: Repetition count along each axis (integer)
- Lattice vectors and atomic positions are expanded accordingly

6.8.10 Move Molecule/Atoms Dialog

Translate selected atoms or molecules.

- **Scope**: "Whole molecule group" or "Selected atoms only"
- **Translation (X, Y, Z)**: Distance in Angstrom for each axis
- **+/- buttons**: Fine adjustment (press-and-hold for continuous operation)
- **Step**: Step size presets (0.01 / 0.1 / 0.5 / 1.0 Angstrom)
- **Apply**: Apply current translation, **Reset**: Return to initial position

6.8.11 Rotate Molecule/Atoms Dialog

Rotate selected atoms or molecules around their center of mass.

- **Scope**: "Whole molecule group" or "Selected atoms only"
- **Trackball**: Drag to rotate interactively
- **alpha(X), beta(Y), gamma(Z)**: Euler angles in degrees
- **+/- buttons**: Fine adjustment (press-and-hold for continuous operation)
- **Step**: Rotation step presets (1 / 5 / 15 / 45 / 90 degrees)
- **Apply**: Apply current rotation, **Reset**: Return to initial orientation

6.8.12 Select by Element Dialog

Select atoms by element using checkboxes.

- Each element shown with a checkbox and atom count
- Selection updates **in real-time** as checkboxes change
- **Mode**: Replace (replace current selection) / Add (add to current selection)

6.8.13 Select by Molecule Dialog

Select atoms by molecule group (mol_id).

- Each group shown with a checkbox, color swatch, and atom count
- **Real-time preview** reflects selection changes immediately
- **Mode**: Replace / Add

6.8.14 Select by Sphere Dialog

Select all atoms within a radius from the currently selected atom.

- **Radius slider**: Drag to adjust radius (real-time preview)
- Shows atom count and element breakdown within the selection

6.8.15 Atom Appearance Panel

Open from **View Settings → Atom Appearance…**. The panel has three tabs:

Color Mode tab:

- **Default (by element)**: Standard jmol element colors
- **By molecule group**: Color by `mol_id` using 20 distinct colors (cycled)

- **By tag**: Color by tag values
- Custom arrays are automatically listed as additional color modes

Element Colors tab:

- Change the color of each element individually
- Click **Change…** to open the color picker
- **Reset to Default** restores jmol default colors

Size tab:

- **Atom Size Scale**: Global atom size multiplier (0.1–3.0, slider)
- **Per-Element Radius Override**: Adjust covalent radius per element

6.8.16 Cell Parameters Dialog

Open from **MoleculeTools → Cell Parameters…**.

- **Lattice lengths**: Display and edit a, b, c (in Å)
- **Lattice angles**: Display and edit α, β, γ (in degrees)
- **Lattice vectors**: Current lattice vectors (read-only)
- **Volume**: Cell volume (Å^3)
- **Apply**: Update cell with the entered values (atomic positions are scaled)

6.8.17 Export Dialog

- Select output directory
- Select output format(s) (QE / VASP / CIF / All)
- Click **OK** to export

CHAPTER

SEVEN

COMMAND-LINE TOOLS

7.1 run.py — Pipeline Execution

Runs the main pipeline.

```
python run.py
```

Processing flow:

1. Build system from `input/input.toml` and `data/*.toml`
2. Index atoms into molecule groups
3. Optimize molecular orientations via Voronoi IoU maximization
4. Check Voronoi volume tiling consistency
5. Render Voronoi cell overlay visualizations → `source/voronoi_pngs/`
6. Write structure files → `source/structure_out/`

Input files (see *Input / Output File Formats* for details):

- `input/input.toml` — Main configuration
- `data/<name>.toml` — Molecule definitions

Output files:

- `source/structure_out/qe.in` — Quantum ESPRESSO input
- `source/structure_out/POSCAR` — VASP structure file
- `source/structure_out/INCAR` — VASP settings
- `source/structure_out/KPOINTS` — VASP k-point settings
- `source/structure_out/structure.cif` — CIF file
- `source/voronoi_pngs/*.png` — Voronoi visualizations

7.2 molecular_modeling_gui.py — GUI Launch

Launches the ASE-GUI extension for molecular modeling. See *GUI Guide* for details.

```
# Launch with empty structure
python molecular_modeling_gui.py

# Launch with a structure file
python molecular_modeling_gui.py [structure_file]
```

Arguments:

Argument	Description
`structure_file` (optional)	Path to any structure file readable by ASE

If no file is specified, a 20x20x20 Angstrom box is created.

7.3 vis_cif.py — CIF Viewer

Displays a CIF file in the GUI.

```
# Open a specific file
python vis_cif.py <path_to_file.cif>

# Open file dialog
python vis_cif.py
```

Arguments:

Argument	Description
`path_to_file.cif` (optional)	Path to CIF file. If omitted, a file chooser dialog opens.

Note: If ASE's CIF parser fails, the tool retries after stripping `_citation*` tags as a fallback.

7.4 edit_and_run.py — GUI Edit + Optimization

Pipeline that allows manual GUI editing before Voronoi optimization.

```
python edit_and_run.py
```

Processing flow:

1. Build system from TOML
2. **Launch GUI** (user edits the structure)
3. After closing the GUI, continue processing with the edited system
4. Build molecule groups
5. Voronoi IoU optimization
6. Visualization and export

7.5 Running Tests

```
# Run all tests
pytest

# Specific test file
pytest tests/test_voronoi_geom.py
```

(continues on next page)

(continued from previous page)

```
# Specific test case
pytest tests/test_voronoi_geom.py::TestPolyhedron::test_volume

# Skip slow tests
pytest -m "not slow"
```

CHAPTER

EIGHT

INPUT / OUTPUT FILE FORMATS

8.1 File Structure Overview

The toolkit uses two types of TOML files:

1. **Input configuration** (`input/input.toml`): Defines supercell lattice, repetition, molecule placement, and substitution rules
2. **Molecule definitions** (`data/<name>.toml`): Defines atomic structure, lattice vectors, and center of mass for each molecule

The relationship between these files is shown below:

```
input/input.toml
├─── molecules = ["benzene", "naphthalene"]
│       ↓                    ↓
│   data/benzene.toml    data/naphthalene.toml
│   (atom coords/lattice) (atom coords/lattice)
│
├─── cell = [...]           ← Supercell lattice vectors
├─── supercell_repeat = [1,1,2]  ← Repetition counts
├─── center_of_mass = [...]      ← Placement positions
└─── [[substitutions]]           ← Substitution rules (optional)
```

Processing flow:

1. Load `input.toml` and read lattice vectors and repetition counts
2. Load `data/<name>.toml` for each entry in the `molecules` list
3. Generate cells for the number of repetitions specified by `supercell_repeat`
4. Place molecules at the `center_of_mass` positions in each cell
5. Apply substitutions (molecule → global element → molecule atom → individual atom)
6. Build ASE `Atoms` object

8.2 Input Configuration: input/input.toml

Main configuration file controlling the pipeline.

8.2.1 Basic Settings

```
# Random seed (integer)
# Used as fallback when optimize/render specific seeds are not set
rng = 1

# Lattice vectors (3x3 array, unit: Angstrom)
# Rows define a, b, c vectors
cell = [
  [ 5.522000,  0.000000,  0.000000 ],   # a-vector
  [ 0.000000,  5.439600,  0.000000 ],   # b-vector
  [-3.501254,  0.000000,  9.339672 ],   # c-vector
]

# Supercell repetition counts [a, b, c directions]
supercell_repeat = [1, 1, 2]

# Molecule names (corresponding to data/<name>.toml files)
molecules = ["benzene", "naphthalene"]

# Whether to use fractional coordinates (true: fractional, false: Cartesian)
use_fractional = false

# Center of mass for each molecule (same order as molecules)
center_of_mass = [
  [  0.000000,   0.000000,   0.000000 ],
  [ -1.750627,   2.719800,   4.669836 ],
]
```

8.2.2 All Keys

Key	Type	Description
`rng`	int	Global random seed. Fallback when `optimize_rng` / `render_rng` are not set.
`optimize_rng`	int (optional)	Random seed for the Voronoi optimization stage. Defaults to `rng` if omitted.
`render_rng`	int (optional)	Random seed for the visualization stage. Defaults to `rng` if omitted.
`cell`	array[3][3]	Lattice vectors (rows: a, b, c, unit: Angstrom)
`supercell_repeat`	array[3]	Supercell repetition counts [nx, ny, nz]
`molecules`	array[str]	Molecule names (corresponding to `data/<name>.toml`)
`use_fractional`	bool	If true, `center_of_mass` is interpreted as fractional coordinates
`center_of_mass`	array[N][3]	Center of mass coordinates for each molecule (same count as `molecules`)

8.2.3 cell_index and Supercell Relationship

When building a supercell with `supercell_repeat = [nx, ny, nz]`, each cell is assigned an index `[ix, iy, iz]`:

- `ix`: 0 to nx-1
- `iy`: 0 to ny-1
- `iz`: 0 to nz-1

For example, with `supercell_repeat = [1, 1, 2]`:

```
Cell [0, 0, 0] — Original unit cell
Cell [0, 0, 1] — Copy shifted by one c-vector
```

Each cell contains the molecules defined in `molecules`, so `molecule_index = 0` refers to benzene and `molecule_index = 1` refers to naphthalene.

The `cell_index` in substitution tables specifies a cell position within this supercell.

8.2.4 Substitution Tables

Four types of substitution tables can be defined using `[[table_name]]` syntax. Substitutions are applied in the following order:

1. **Molecule substitutions** (`molecule_substitutions`): Replace entire molecules
2. **Global atom substitutions** (`global_atom_substitutions`): Replace all atoms of an element
3. **Molecule atom substitutions** (`molecule_atom_substitutions`): Replace atom list of a specific molecule
4. **Individual atom substitutions** (`atom_substitutions`): Replace a single atom

molecule_substitutions

Replace a molecule at a specific cell position with a different molecule.

```toml
[[molecule_substitutions]]
cell_index       = [0, 0, 1]       # Cell position in supercell
molecule_index   = 1               # Molecule index (0-based)
new_molecule     = "benzene"       # Replacement molecule name
```

Key	Type	Description
`cell_index`	array[3]	Cell position in supercell [ix, iy, iz]
`molecule_index`	int	Molecule index within cell (0-based)
`new_molecule`	str	Replacement molecule name

global_atom_substitutions

Replace all atoms of a specified element throughout the system.

```toml
[[global_atom_substitutions]]
old_symbol = "H"      # Original element symbol
new_symbol = "He"     # Replacement element symbol
```

Key	Type	Description
`old_symbol`	str	Original element symbol
`new_symbol`	str	Replacement element symbol

molecule_atom_substitutions

Replace the atom list of a specific molecule instance with a new list.

> **Note**
>
> This substitution overwrites the entire atom list, so any `global_atom_substitutions` applied to the same molecule will be overridden.

```
[[molecule_atom_substitutions]]
cell_index     = [0, 0, 1]
molecule_index = 1
new_symbols    = ["Si6O6"]    # Compact notation supported
```

Key	Type	Description
`cell_index`	array[3]	Cell position in supercell
`molecule_index`	int	Molecule index within cell
`new_symbols`	array[str]	New element symbol list (compact notation supported)

atom_substitutions

Replace a single specific atom.

```
[[atom_substitutions]]
cell_index     = [0, 0, 0]
molecule_index = 1
atom_index     = 2           # Atom index within molecule (0-based)
new_symbol     = "Fe"
```

Key	Type	Description
`cell_index`	array[3]	Cell position in supercell
`molecule_index`	int	Molecule index within cell
`atom_index`	int	Atom index within molecule (0-based)
`new_symbol`	str	Replacement element symbol

8.3 Molecule Definition: data/<name>.toml

Files defining the structure information for each molecule. The file name (without `.toml` extension) must match the names listed in `molecules` in `input.toml`.

> **Note**
>
> The `center_of_mass` in `data/*.toml` is used as **molecule dictionary reference data**. The actual placement position in the supercell is determined by `center_of_mass` in `input.toml`.
>
> The `lattice_vectors` in `data/*.toml` is the **molecule's own crystal lattice**. The overall supercell lattice is defined by `cell` in `input.toml`. For mixed crystals (co-crystals of different molecules), these will have different values.

8.3.1 Basic Structure

```toml
molecule_name = "benzene"

# Element symbols: individual list or compact notation
symbols = ["C", "C", "C", "C", "C", "C", "H", "H", "H", "H", "H", "H"]
# or: symbols = ["C6H6"]

# Coordinate system
use_fractional = false

# Atomic coordinates (Nx3 array)
positions = [
  [-1.113710, -0.097913, -0.811833],
  [ 1.113710,  0.097913,  0.811833],
  # ...
]

# Lattice vectors (3x3 array, unit: Angstrom)
lattice_vectors = [
  [ 5.522000,  0.000000,  0.000000],
  [ 0.000000,  5.439600,  0.000000],
  [-2.693272,  0.000000,  7.184363],
]

# Number of molecules per unit cell
num_molecules = 2

# Center of mass for each molecule
center_of_mass = [
  [ 0.000000,  0.000000,  0.000000],
  [-1.346636,  2.719800,  3.592182],
]

# Voronoi reference molecule index (default: 0)
# voronoi_ref_id = 0
```

8.3.2 All Keys

Key	Type	Required	Description
`molecule_name`	str	Yes	Molecule identifier
`symbols`	array[str]	Yes	Element symbols. Compact notation supported (e.g., `"C6H6"`)
`use_fractional`	bool	No	If true, positions and center_of_mass are fractional coordinates
`positions`	array[N][3]	Yes	Atomic coordinates (Cartesian Angstrom or fractional)
`lattice_vectors`	array[3][3]	Yes	Lattice vectors (rows: a, b, c)
`num_molecules`	int	Yes	Number of molecules per unit cell
`center_of_mass`	array[K][3]	Yes	Center of mass for K molecules
`voronoi_ref_id`	int	No	Reference molecule index for Voronoi cell computation (default: 0)

8.3.3 Compact Notation

The `symbols` field supports concatenated element-count notation:

```
"C6H6"    → ["C", "C", "C", "C", "C", "C", "H", "H", "H", "H", "H", "H"]
"C10H8"   → ["C"] x 10 + ["H"] x 8
"Si6O6"   → ["Si"] x 6 + ["O"] x 6
```

8.3.4 Coordinate Conversion

When `use_fractional = true`, the following conversion is applied internally:

```
pos_cart = pos_frac @ lattice_vectors
pos_frac = pos_cart @ inv(lattice_vectors)
```

Note

The `use_fractional` flags in `input.toml` and `data/*.toml` are **independent**. One can use fractional coordinates while the other uses Cartesian coordinates.

8.4 Complete Configuration Examples

Below are the complete configuration examples for the included benzene-naphthalene mixed crystal.

8.4.1 input/input.toml Complete Example

```
# Random seed
rng = 1

# Supercell lattice vectors (Angstrom)
# Lattice for benzene-naphthalene mixed crystal
```

(continues on next page)

(continued from previous page)

```
cell = [
  [ 5.522000,  0.000000,  0.000000 ],
  [ 0.000000,  5.439600,  0.000000 ],
  [-3.501254,  0.000000,  9.339672 ],
]

# Build a supercell doubled along the c-axis
supercell_repeat = [1, 1, 2]

# Use two molecule types
# References data/benzene.toml and data/naphthalene.toml
molecules = ["benzene", "naphthalene"]

# Use Cartesian coordinates
use_fractional = false

# Placement position for each molecule (same order as molecules)
# benzene → origin, naphthalene → offset position
center_of_mass = [
  [  0.000000,   0.000000,   0.000000 ],
  [ -1.750627,   2.719800,   4.669836 ],
]

# --- Optional substitution rules below ---

# Replace the 2nd molecule (naphthalene) in cell [0,0,1] with benzene
[[molecule_substitutions]]
cell_index       = [0, 0, 1]
molecule_index   = 1
new_molecule     = "benzene"

# Replace all H with He
[[global_atom_substitutions]]
old_symbol = "H"
new_symbol = "He"

# Replace atom list of the 2nd molecule in cell [0,0,1] with Si6O6
[[molecule_atom_substitutions]]
cell_index     = [0, 0, 1]
molecule_index = 1
new_symbols    = ["Si6O6"]

# Replace the 3rd atom of the 2nd molecule in cell [0,0,0] with Fe
[[atom_substitutions]]
cell_index     = [0, 0, 0]
molecule_index = 1
atom_index     = 2
new_symbol     = "Fe"
```

8.4.2 data/benzene.toml Complete Example

This example uses the **individual list** format for element symbols.

```
molecule_name = "benzene"

# 12 atoms listed individually
symbols = [
  "C", "C", "C", "C", "C", "C",
  "H", "H", "H", "H", "H", "H",
]

use_fractional = false

# Atomic coordinates of the 1st molecule (Angstrom)
# Corresponds to symbols in the same order
positions = [
  # Carbon atoms (6)
  [ -1.113710, -0.097913, -0.811833 ],
  [  1.113710,  0.097913,  0.811833 ],
  [ -0.115564, -1.087920, -0.865716 ],
  [  0.115564,  1.087920,  0.865716 ],
  [ -0.979384,  0.951930,  0.052446 ],
  [  0.979384, -0.951930, -0.052446 ],
  # Hydrogen atoms (6)
  [ -1.861055, -0.158292, -1.362155 ],
  [  1.861055,  0.158292,  1.362155 ],
  [ -0.197159, -1.816826, -1.437591 ],
  [  0.197159,  1.816826,  1.437591 ],
  [ -1.651612,  1.593259,  0.089805 ],
  [  1.651612, -1.593259, -0.089805 ],
]

# Benzene crystal lattice vectors
lattice_vectors = [
  [ 5.522000,  0.000000,  0.000000],
  [ 0.000000,  5.439600,  0.000000],
  [-2.693272,  0.000000,  7.184363],
]

# 2 molecules per unit cell
num_molecules = 2

# Center of mass for each molecule
center_of_mass = [
  [ 0.000000,  0.000000,  0.000000],
  [-1.346636,  2.719800,  3.592182],
]
```

8.4.3 data/naphthalene.toml Complete Example

This example uses **compact notation** for element symbols.

```
molecule_name = "naphthalene"

# Compact notation: C10H8 → C x 10 + H x 8 = 18 atoms
symbols = ["C10H8"]

use_fractional = false

# Atomic coordinates of the 1st molecule (Angstrom)
# Corresponds to expanded symbols (C first, then H)
positions = [
  # Carbon atoms (10)
  [ -0.873883,  0.110730,  2.365899 ],
  [  0.873883, -0.110730, -2.365899 ],
  [ -0.125794,  0.966533,  1.590346 ],
  [  0.125794, -0.966533, -1.590346 ],
  [  0.213579,  0.617262,  0.251858 ],
  [ -0.213579, -0.617262, -0.251858 ],
  [  0.981797,  1.483077, -0.575778 ],
  [ -0.981797, -1.483077,  0.575778 ],
  [ -1.309714, -1.131450,  1.848626 ],
  [  1.309714,  1.131450, -1.848626 ],
  # Hydrogen atoms (8)
  [ -1.094109,  0.351038,  3.237058 ],
  [  1.094109, -0.351038, -3.237058 ],
  [  0.159973,  1.778750,  1.942805 ],
  [ -0.159973, -1.778750, -1.942805 ],
  [  1.263671,  2.303540, -0.240443 ],
  [ -1.263671, -2.303540,  0.240443 ],
  [ -1.818179, -1.705126,  2.375888 ],
  [  1.818179,  1.705126, -2.375888 ],
]

# Naphthalene crystal lattice vectors (different from benzene)
lattice_vectors = [
  [ 8.034800,  0.000000,  0.000000],
  [ 0.000000,  5.889900,  0.000000],
  [-4.738553,  0.000000,  7.134798],
]

# 2 molecules per unit cell
num_molecules = 2

# Center of mass for each molecule
center_of_mass = [
  [ 0.000000,  0.000000,  0.000000],
  [-1.346636,  2.719800,  3.592182],
]
```

8.4.4 Creating New Molecule Definitions

To add a custom molecule, create a `data/<name>.toml` file with the following steps:

1. Set `molecule_name` to the molecule name
2. Define element symbols in `symbols` (individual list or compact notation)
3. Write the **1st molecule's** atomic coordinates in `positions` (center of mass near the origin)
4. Set the molecule's crystal lattice in `lattice_vectors`
5. Set the number of molecules per unit cell in `num_molecules`
6. Write the center of mass for each molecule in `center_of_mass`

Tip

You can use **MoleculeTools → Save Selection as Molecule…** in the GUI to save selected atoms from the 3D view as a `data/*.toml` file.

APPENDIX

A

VORONOI OPTIMIZATION DETAILS

This appendix explains the Voronoi cell IoU optimization algorithm, a core feature of the toolkit, in detail.

A.1 Problem Setting

When modeling organic molecular crystals and molecular solids, it is necessary to appropriately determine the **orientation** (direction) of molecules placed in a unit cell. This toolkit uses the centers of mass of molecules as generator points for Voronoi tessellation and automatically determines optimal orientations by maximizing the **geometric overlap** between the Voronoi cells of the system and those of the dictionary (reference crystal).

Here, "system" refers to the supercell actually constructed, and "dictionary" refers to the reference single-crystal structure.

A.2 What Are Voronoi Cells?

When Voronoi tessellation is performed using the center of mass of each molecular site in the crystal as a generator point, space is divided into regions that are "closest" to each site. These regions are Voronoi cells (Voronoi polyhedra).

```
Properties of Voronoi cells:
- One convex polyhedron corresponds to each generator point
- Sum of all cell volumes = unit cell volume (space is filled without gaps)
- Cell shapes are determined by relative positions between molecules
```

This toolkit performs Voronoi tessellation with periodic boundary conditions. Specifically, it generates images for ±1 cell (27 periodic images) and then computes Voronoi cell vertices using SciPy's halfspace intersection method (HalfspaceIntersection).

A.3 IoU (Intersection over Union)

IoU is a metric that represents the degree of overlap between two solids, taking values from 0 to 1.

$$\text{IoU} = \frac{\text{Intersection Volume (Overlap)}}{\text{V_sys} + \text{V_dict} - \text{Intersection Volume}}$$

```
IoU = 1: Two Voronoi cells match perfectly
IoU = 0: Two Voronoi cells do not overlap at all
```

Where:

- **V_sys**: Voronoi cell volume of the system (supercell)
- **V_dict**: Voronoi cell volume of the dictionary (reference crystal, after scaling)

- **Intersection volume**: Volume of the overlapping region of the two cells

The intersection volume is computed by performing Voronoi tessellation on the combined generator points of both the system and dictionary, and obtaining the volume of the combined cell for the reference point. This corresponds to the intersection volume of V_sys and V_dict.

A.4 Per-Site Optimization Procedure

Optimization is performed independently for each molecular site. Each site undergoes the following 5 steps.

A.4.1 Step 1: System Voronoi Cell Volume Calculation

Using all molecular centers of mass in the supercell as generator points, the Voronoi cell volume V_sys at the target site's center of mass is computed.

```
Input: Supercell lattice A_sys, all molecular center-of-mass coordinates
Output: V_sys (angstrom^3)
```

The system's Voronoi cell does not change during optimization (because center-of-mass positions remain unchanged even when orientations change).

A.4.2 Step 2: Dictionary Voronoi Cell Volume Calculation

Using the lattice vectors and centers of mass from the dictionary (reference crystal), the dictionary-side Voronoi cell volume V_dict is computed.

```
Input: Dictionary lattice A_dict, dictionary center-of-mass coordinates
Output: V_dict (angstrom^3)
```

A.4.3 Step 3: Volume Scaling

When the system and dictionary Voronoi cell volumes differ (e.g., in mixed crystals), uniform scaling is applied to the dictionary's lattice and centers of mass to match the volumes.

```
Scale factor: s = (V_sys / V_dict)^(1/3)
```

This scaling makes the dictionary Voronoi cell volume equal to that of the system. The cube root is used because in 3D uniform scaling, the volume scales as s^3.

A.4.4 Step 4: Euler Angle Optimization

The dictionary Voronoi cell is rotated to find Euler angles (alpha, beta, gamma) that maximize IoU with the system Voronoi cell.

Coarse Grid Search (Phase 1):

```
alpha, beta, gamma each divided into 13 steps over [-180 deg, 180 deg]
-> 13 x 13 x 13 = 2,197 combinations evaluated
-> The angles with the highest IoU are recorded as the initial solution
```

Nelder-Mead Refinement (Phase 2):

```
Refine using the Nelder-Mead method with the best grid search result as␣
↪initial value
- Objective function: f(alpha, beta, gamma) = 1 - IoU (minimization)
```

(continues on next page)

(continued from previous page)

```
- Angle tolerance: 0.001 deg
- IoU tolerance: 0.001
- Maximum iterations: 2,000
- Maximum function evaluations: 10,000
```

The Nelder-Mead method is a derivative-free direct search method, well-suited for objective functions like IoU where derivatives are difficult to compute.

Euler angle rotation convention:

```
Rotation order: XYZ (extrinsic)
- alpha: rotation about X axis
- beta: rotation about Y axis
- gamma: rotation about Z axis

Rotation matrix R = R_z(gamma) x R_y(beta) x R_x(alpha)
```

A.4.5 Step 5: Application to Atomic Coordinates

The optimized rotation angles and scale are applied to all atomic coordinates of the target site.

```
New coordinates for each atom:
  pos_new = p_com + s x R x (pos_old - p_com)

p_com: Center of mass of the site
s:     Scale factor (Step 3)
R:     Optimal rotation matrix (Step 4)
```

Since rotation and scaling are performed about the center of mass, the molecular center-of-mass positions remain unchanged.

A.5 Dictionary Voronoi Cell Construction

The dictionary Voronoi cell is constructed by the following procedure:

1. Translate dictionary centers of mass so that the reference molecule (`voronoi_ref_id`, default 0) is at the origin
2. Rotate by Euler angles (alpha, beta, gamma)
3. Uniformly scale by factor s
4. Translate to the center-of-mass position of the target site in the system
5. Apply the same rotation and scale to the dictionary lattice and generate images for ±1 cell
6. Perform combined Voronoi tessellation from all generator points of both system and dictionary

This procedure places the dictionary Voronoi cell in the system's coordinate space, enabling IoU computation.

A.6 Volume Tiling Verification

After optimization, the toolkit verifies that the total Voronoi cell volumes match the supercell volume.

```
Verification condition:
  |sum(V_i) - V_cell| <= abs_tol  or  |sum(V_i) - V_cell| / V_cell <= rel_tol

Default tolerances:
  abs_tol = 1e-6 angstrom^3
  rel_tol = 1e-3 (0.1%)
```

If this verification fails, there may be a problem with the Voronoi tessellation (e.g., molecules placed too closely, extremely distorted lattice).

A.7 Interpreting Optimization Results

Optimization results are stored in `system.info` and can be viewed in the GUI via **Pipeline** > **Show Optimization Results**.

Output Item	Meaning
IoU	Degree of overlap between system and dictionary Voronoi cells (0–1). Closer to 1 means the dictionary molecular orientation is better reproduced in the system
alpha, beta, gamma	Optimized Euler angles (degrees). Amount of rotation relative to the dictionary molecular orientation
scale	Volume scaling factor. 1.0 means the system and dictionary Voronoi volumes are equal. Greater than 1 means the system site is larger than the dictionary

IoU Guidelines:

- **0.9 or above**: The dictionary orientation is very well reproduced in the system
- **0.7 to 0.9**: Generally good reproduction
- **Below 0.7**: Significant differences between the system and dictionary environments (e.g., effects of different molecules in mixed crystals)

A.8 Visualization (Voronoi Overlay PNG)

Optimization results are visualized as 3D images:

- **Blue semi-transparent polyhedron**: System Voronoi cell (actual spatial partitioning)
- **Red semi-transparent polyhedron**: Dictionary Voronoi cell (reference cell rotated and scaled by optimization)
- **Orange semi-transparent polyhedron**: Intersection of the two cells

The higher the IoU, the more the blue and red polyhedra overlap, becoming nearly identical in shape.

A.9 Libraries Used

Library	Purpose
SciPy (`scipy.spatial`)	Voronoi tessellation, halfspace intersection, convex hull computation, Nelder-Mead optimization
SciPy (`scipy.spatial.transform`)	Rotation matrix generation from Euler angles
trimesh	Polyhedral mesh construction, volume computation, intersection calculations
manifold3d	Polyhedral boolean operations (intersection, union)
NumPy	Numerical array operations, linear algebra

APPENDIX

B

ARCHITECTURE

B.1 Pipeline Flow

The main pipeline (`run.py` → `source/main.py`) processes in the following order:

```
input/input.toml + data/*.toml
       |
       v
+-------------------------+
|  build_system_from_toml |  TOML -> ASE Atoms construction
|                         |  Supercell generation, substitutions
+----------+--------------+
           |  system (Atoms + system.info)
           v
+-------------------------+
|  build_molecule_groups  |  Atom grouping by mol_id
+----------+--------------+
           |  groups: list[list[int]]
           v
+-------------------------+
| optimize_voronoi_overlap|  Euler angle grid search +
|                         |  Nelder-Mead optimization
+----------+--------------+
           |  optimized_angles_deg, scales, iou
           v
+-------------------------------+
| check_system_voronoi_volume   |  Voronoi volume
| _tiling (voronoi_geom.py)     |  tiling verification
+----------+--------------------+
           |
           v
+-------------------------+
| render_voronoi_overlays |  3D matplotlib visualization
+----------+--------------+
           |  voronoi_pngs/*.png
           v
+-------------------------+
|     write_structure     |  QE / VASP / CIF export
+----------+--------------+
           |
           v
structure_out/  (qe.in, POSCAR, INCAR, KPOINTS, structure.cif)
```

B.2 Module Descriptions

B.2.1 build_system_from_toml.py

Input: `input/input.toml`, `data/*.toml`

Output: ASE `Atoms` object

Main operations:

1. Read and parse TOML files
2. Convert fractional → Cartesian coordinates (when `use_fractional` is true)
3. Build supercell (repetition tiling)
4. Place each molecule (translation to center-of-mass position)
5. Apply 4 types of substitution tables
6. Generate `mol_id` array (maps each atom to its site)
7. Set metadata in `system.info`

B.2.2 build_molecule_groups.py

Input: `Atoms` object (with `mol_id` array)

Output: `list[list[int]]` — atom index list per site

Scans the `mol_id` array and groups atom indices with the same value.

B.2.3 voronoi_geom.py

The core geometry module, providing:

- **Polyhedron class**: Convex polyhedron representation (vertices + faces)
- **Voronoi cell computation**: Periodic Voronoi tessellation via SciPy halfspace intersection
- **IoU computation**: Volume-based Intersection over Union using trimesh and manifold3d
- **Rotation and scaling**: Polyhedron rotation by Euler angles and uniform scaling
- **Periodic boundary handling**: Wrapping at cell boundaries

B.2.4 optimize_voronoi_overlap.py

Process: Search for Euler angles that maximize Voronoi IoU per site

Algorithm:

1. Compute system Voronoi cell volume `V_sys`
2. Compute dictionary Voronoi cell volume `V_dict`
3. Set uniform scale `s = (V_sys / V_dict)^(1/3)`
4. Coarse grid search over Euler angles (alpha, beta, gamma) (default 13x13x13)
5. Refine from best point using Nelder-Mead
6. Apply optimal rotation and scaling to atom positions

Results are stored in `system.info`:

- `optimized_angles_deg`: Optimal Euler angles (Nx3)
- `optimized_scales`: Optimal scales (N,)

- `optimized_iou`: IoU values (N,)

B.2.5 render_voronoi_overlays.py

Renders 3D matplotlib plots overlaying system and dictionary Voronoi cells.

- Blue translucent: System Voronoi cell
- Red translucent: Dictionary Voronoi cell
- One PNG per site

B.2.6 write_structure.py

Uses ASE I/O to write structure files in the following formats:

- **Quantum ESPRESSO**: `qe.in` (with SCF calculation settings)
- **VASP**: `POSCAR`, `INCAR`, `KPOINTS`
- **CIF**: `structure.cif`

B.3 system.info Metadata

The `info` dictionary on ASE `Atoms` objects is used to pass data between pipeline stages.

Key	Type	Description
`groups`	list[list[int]]	Atom indices per site
`site_mol_name`	list[str]	Molecule name per site
`dict_meta`	dict	Per-molecule metadata (lattice vectors, COMs, voronoi_ref_id)
`rng`	dict	Random seeds (global, optimize, render)
`optimized_angles_deg`	ndarray(N,3)	Optimized Euler angles (degrees)
`optimized_scales`	ndarray(N,)	Optimized scale factors
`optimized_iou`	ndarray(N,)	Optimized IoU values

B.3.1 Atom Arrays

Array	Type	Description
`mol_id`	int[N_atoms]	Site index for each atom

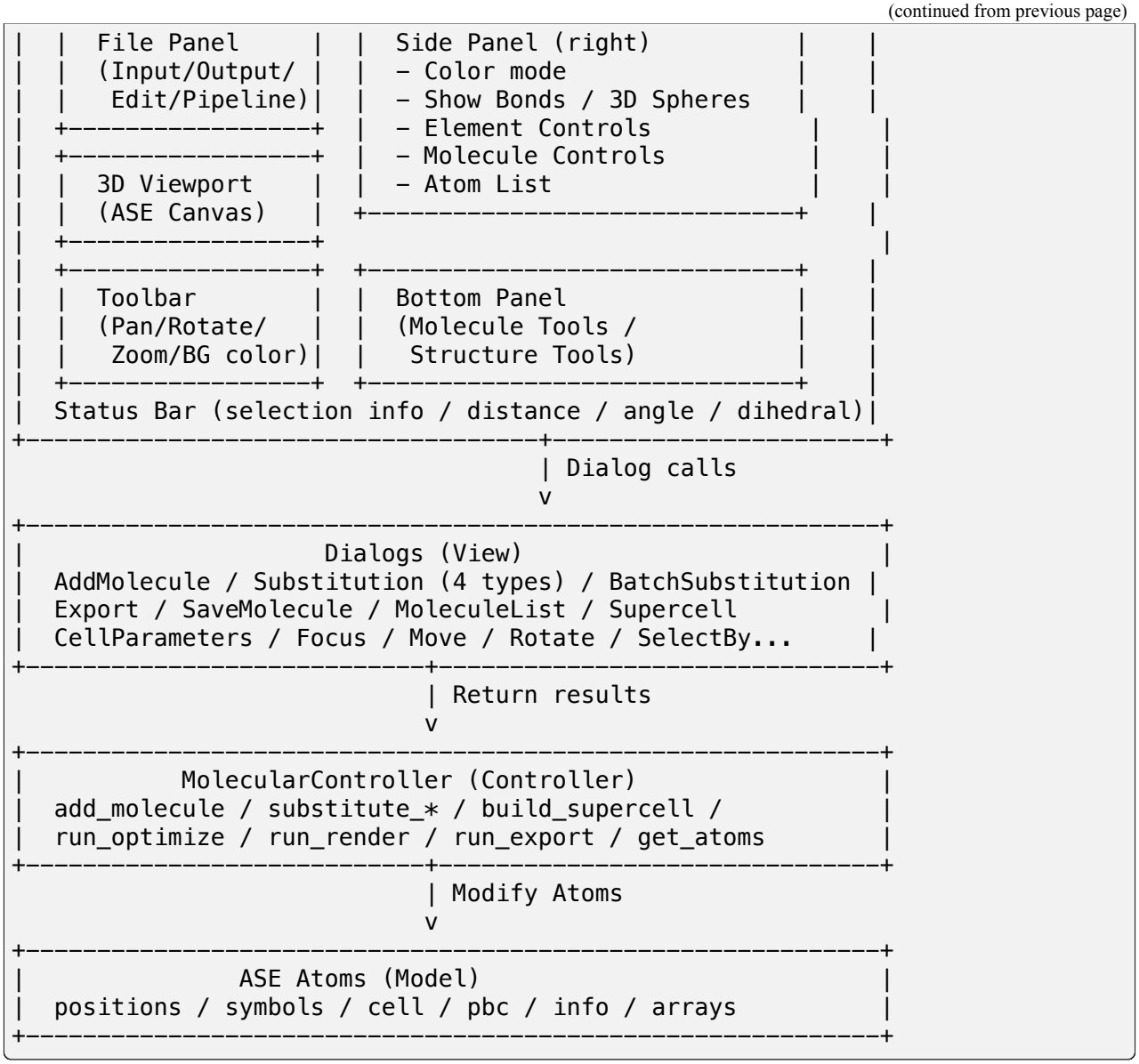
B.4 GUI Architecture

The GUI follows an MVC (Model-View-Controller) pattern.

```
+-------------------------------------------------------------+
|              MolecularModelingGUI (View)                    |
|                                                             |
|  Menus: MoleculeTools / Recent Files / Export /             |
|         Selection / Pipeline / View Settings / Help         |
|                                                             |
|  +-----------------+  +-----------------------------+    |
```

(continues on next page)

(continued from previous page)



GUI module structure:

- `molecular_modeling_gui.py`: Main GUI class (extends ASE-GUI). Manages window layout, menus, toolbar, side panel, and status bar
- `source/gui/controller.py`: Bridge between GUI dialogs and backend operations
- `source/gui/theme.py`: Theme configuration (colors, fonts, spacing, ttk theme setup)
- `source/gui/base_dialog.py`: Base class for modal dialogs (OK/Cancel, validation, coordinate input)
- `source/gui/molecule_registry.py`: Dynamic molecule definition loader from `data/*.toml`
- `source/gui/add_molecule_dialog.py`: Add molecule dialog
- `source/gui/substitution_dialogs.py`: Four substitution dialog types + batch element substitution
- `source/gui/export_dialog.py`: Export dialog
- `source/gui/save_molecule_dialog.py`: Save selected atoms as molecule dialog
- `source/gui/molecule_list_dialog.py`: Molecule list dialog

- `source/gui/supercell_dialog.py`: Supercell settings dialog

APPENDIX

C

API REFERENCE

This section provides auto-generated API documentation for the main modules.

C.1 Module Organization

The toolkit consists of the following module groups:

- **Pipeline**: `main` is the entry point. Processing flows through `build_system_from_toml` → `build_molecule_groups` → `optimize_voronoi_overlap` → `render_voronoi_overlays` → `write_structure`
- **Geometry**: `voronoi_geom` handles Voronoi cell construction, volume calculation, and IoU evaluation. The `Polyhedron` class is the core abstraction
- **GUI**: `gui.controller` bridges views (dialogs) and the model (ASE Atoms). `gui.molecule_registry` manages dynamic loading of `data/*.toml`

Basic programmatic usage:

```
from molecular_modeling.main import main

# Read input/input.toml and run the full pipeline
main()
```

C.2 Main Pipeline

C.2.1 main

`molecular_modeling.main.main()`

Entry point: build system, optimize orientations, render, and export structures.

Pipeline

1. Build *Atoms* from TOML inputs.
2. Group atoms by molecule/site.
3. Optimize Voronoi IoU via rotations (per site).
4. Check Voronoi volume tiling consistency.
5. Render overlays to PNGs.
6. Write QE/VASP/CIF outputs.

C.3 System Construction

C.3.1 build_system_from_toml

molecular_modeling.build_system_from_toml.**build_system_from_toml**(** (Keyword-only parameters separator (PEP 3102))*, *verbose: bool = True*) → Atoms

Build an ASE *Atoms* object from TOML inputs (supercell + substitutions).

Parameters
verbose (*bool, optional*) – If True, print a summary of the assembled system.

Returns
Periodic supercell with per-atom *mol_id* and metadata in *system.info* (*groups*, *site_mol_name*, *dict_meta*, *rng*).

Return type
ase.Atoms

Notes

- **Input (*input/input.toml*)**
  - *use_fractional=true* interprets *center_of_mass* as fractional and converts to Cartesian via *cell* (rows a,b,c).
  - Downstream code always uses Cartesian site COMs.
- **Dictionary (*data/<name>.toml*)**
  - *voronoi_ref_id* selects which molecule COM is the reference (0-based).
  - If *use_fractional=true*, both *positions* and *center_of_mass* are converted to Cartesian with that dictionary's *lattice_vectors*.
  - *dict_meta* stores dictionary COMs in Cartesian.
- **Placement**
  - For each site: recenter *positions* by its COM, rotate (initially zero), then translate to the site COM; lattice tiling handled by *supercell_repeat*.

C.3.2 build_molecule_groups

molecular_modeling.build_molecule_groups.**build_molecule_groups**(*system: Atoms*) → List[List[int]]

Build atom index groups keyed by the *mol_id* array.

Parameters
system (*ase.Atoms*) – System holding *system.arrays['mol_id']*, where each value marks the placement/site identifier for an atom.

Returns
List of atom index lists, ordered by *mol_id* (0..K-1).

Return type
list[list[int]]

Raises
ValueError – If *mol_id* is missing from *system.arrays*.

C.4 Voronoi Geometry

C.4.1 voronoi_geom

class molecular_modeling.voronoi_geom.**Polyhedron**(*verts: ndarray*, *faces: ndarray*)

Bases: object

Minimal container for a convex polyhedron.

Parameters
- **verts** (*ndarray*) – Vertices array with shape (V, 3).
- **faces** (*ndarray*) – Faces as indices into *verts* (F,3) or (F,k>=3).

verts

Vertex coordinates (float64).

Type
ndarray

faces

Face indices (int).

Type
ndarray

molecular_modeling.voronoi_geom.**polyhedron_to_trimesh**(*poly: Polyhedron*) → Trimesh

Convert *Polyhedron* to a processed *trimesh.Trimesh*.

Parameters
poly (*Polyhedron*) – Input polyhedron.

Returns
Processed mesh with triangulated faces.

Return type
trimesh.Trimesh

molecular_modeling.voronoi_geom.**compute_voronoi_cells_from_coms**(*cell: ndarray*, *coms_cart: ndarray*) → List[Polyhedron]

Compute periodic Voronoi cells for COM points and clip to the unit cell.

Parameters
- **cell** (*ndarray*) – Lattice vectors (3,3) with rows a, b, c (Cartesian Å).
- **coms_cart** (*ndarray*) – COM coordinates (N,3) in Cartesian.

Returns
Voronoi cells corresponding to each COM (same order).

Return type
list[Polyhedron]

molecular_modeling.voronoi_geom.**rotate_and_place_polyhedron**(*poly: Polyhedron*, *euler_deg: Sequence[float]*, *src_center: ndarray*, *dst_center: ndarray*) → Polyhedron

Rotate a polyhedron about one center and translate to another.

Parameters

- **poly** (*Polyhedron*) – Polyhedron to transform.
- **euler_deg** (*sequence[float]*) – Euler angles (XYZ, degrees).
- **src_center** (*ndarray*) – Rotation center (3,).
- **dst_center** (*ndarray*) – Target center after translation (3,).

Returns
Rotated and translated polyhedron.

Return type
Polyhedron

molecular_modeling.voronoi_geom.**compute_system_voronoi_cells**(*system: Atoms*, *groups: List[List[int]]*) → List[Polyhedron]

Compute Voronoi cells for system sites using atom groups.

Parameters

- **system** (*ase.Atoms*) – System with positions and lattice.
- **groups** (*list[list[int]]*) – Atom indices per site.

Returns
Voronoi cells per site (same order as *groups*).

Return type
list[Polyhedron]

molecular_modeling.voronoi_geom.**rotate_polyhedron**(*poly: Polyhedron*, *euler_deg: Sequence[float]*, *center: ndarray*) → Polyhedron

Rotate a polyhedron about a specified center.

Parameters

- **poly** (*Polyhedron*) – Polyhedron to rotate.
- **euler_deg** (*sequence[float]*) – Euler angles (XYZ, degrees).
- **center** (*ndarray*) – Rotation center (3,).

Returns
Rotated polyhedron.

Return type
Polyhedron

molecular_modeling.voronoi_geom.**wrap_angles_deg**(*x: ndarray*) → ndarray

Wrap Euler angles to `[-180, 180)` for numerical stability.

Parameters
x (*ndarray*) – Angles in degrees.

Returns
Wrapped angles.

Return type
ndarray

molecular_modeling.voronoi_geom.**polyhedron_volume**(*poly: Polyhedron*) → float

Compute absolute volume of a closed polygonal mesh via tetrahedra sum.

Parameters
poly (*Polyhedron*) – Polyhedron with vertices and faces.

Returns
Positive volume in Å^3.

Return type
float

molecular_modeling.voronoi_geom.**overlap_volume_union_cart**(*A_sys: ndarray*, *coms_sys_cart: ndarray*, *A_dict: ndarray*, *coms_dict_cart: ndarray*, *angles_deg: Sequence[float]*, *p_ref_cart: ndarray*, *rng: int = 1*, *scale: float = 1.0*, *ref_idx: int = 0*) → float

Compute overlap volume between system and dictionary Voronoi cells.

Parameters

- **A_sys** (*ndarray*) – System lattice (3,3).
- **coms_sys_cart** (*ndarray*) – System COMs (N,3).
- **A_dict** (*ndarray*) – Dictionary lattice (3,3).
- **coms_dict_cart** (*ndarray*) – Dictionary COMs (K,3).
- **angles_deg** (*sequence[float]*) – Euler angles (XYZ, degrees) applied to dictionary.
- **p_ref_cart** (*ndarray*) – Reference system point (3,).
- **rng** (*int, optional*) – Replication radius.
- **scale** (*float, optional*) – Uniform scaling of dictionary lattice/COMs.
- **ref_idx** (*int, optional*) – Reference COM index in the dictionary.

Returns
Overlap volume in Å^3.

Return type
float

molecular_modeling.voronoi_geom.**single_set_cell_volume_cart**(*A_cell: ndarray*, *coms_cart: ndarray*, *p_ref_cart: ndarray*, *which: str*, *angles_deg: Sequence[float] | None = None*, *rng: int = 1*, *scale: float = 1.0*, *ref_idx: int = 0*) → float

Compute Voronoi cell volume for either system or dictionary seeds.

Parameters

- **A_cell** (*ndarray*) – Lattice vectors (3,3).
- **coms_cart** (*ndarray*) – COMs (N,3).
- **p_ref_cart** (*ndarray*) – Reference point (3,).
- **which** (*{'system', 'dict'}*) – Whether to treat seeds as system or dictionary.
- **angles_deg** (*sequence[float], optional*) – Euler angles for dictionary case.
- **rng** (*int, optional*) – Replication radius.
- **scale** (*float, optional*) – Uniform scaling (dictionary case).
- **ref_idx** (*int, optional*) – Reference COM index (dictionary case).

Returns
Voronoi cell volume in Å^3.

Return type
float

Raises
ValueError – If *which* is not 'system' or 'dict'.

molecular_modeling.voronoi_geom.**iou_via_union_cart**(*A_sys: ndarray*, *coms_sys_cart: ndarray*, *A_dict: ndarray*, *coms_dict_cart: ndarray*, *p_ref_cart: ndarray*, *angles_deg: Sequence[float]*, *rng: int = 1*, *scale: float = 1.0*, *ref_idx: int = 0*) → float

Compute IoU between system and dictionary Voronoi cells at a reference site.

Parameters

- **A_sys** (*ndarray*) – System lattice (3,3).
- **coms_sys_cart** (*ndarray*) – System COMs (N,3).
- **A_dict** (*ndarray*) – Dictionary lattice (3,3).
- **coms_dict_cart** (*ndarray*) – Dictionary COMs (K,3).
- **p_ref_cart** (*ndarray*) – Reference system point (3,).
- **angles_deg** (*sequence[float]*) – Euler angles (XYZ, degrees) for dictionary rotation.
- **rng** (*int, optional*) – Replication radius.
- **scale** (*float, optional*) – Uniform scaling for dictionary.
- **ref_idx** (*int, optional*) – Reference COM index in the dictionary.

Returns
Intersection-over-union in [0,1].

Return type
float

molecular_modeling.voronoi_geom.**optimize_angles_each_system_site**(*A_sys: ndarray*, *coms_sys_cart: ndarray*, *A_dict: ndarray*, *coms_dict_cart: ndarray*, *sites_sys_cart: ndarray*, *angles_init: Sequence[float] = (0.0, 0.0, 0.0)*, *rng: int = 1*, *optimizer: str = 'Powell'*) → List[tuple]

Optimize dictionary orientation for each system site independently.

Parameters

- **A_sys** (*ndarray*) – System lattice (3,3).
- **coms_sys_cart** (*ndarray*) – System COMs (N,3).
- **A_dict** (*ndarray*) – Dictionary lattice (3,3).
- **coms_dict_cart** (*ndarray*) – Dictionary COMs (K,3).
- **sites_sys_cart** (*ndarray*) – Reference system sites (N,3).
- **angles_init** (*sequence[float], optional*) – Initial Euler angles (XYZ, degrees) for all sites.
- **rng** (*int, optional*) – Replication radius.
- **optimizer** (*str, optional*) – SciPy optimizer method (passed to *minimize*).

Returns

Tuples of `(site_index, angles_opt, best_iou)`.

Return type

list[tuple]

molecular_modeling.voronoi_geom.**check_system_voronoi_volume_tiling**(*system: Atoms*, *groups: List[List[int]] | None = None*, ***, *rel_tol: float = 0.001*, *abs_tol: float = 1e-06*, *verbose: bool = True*) → dict

Check whether Voronoi cell volumes tile the supercell volume.

Parameters

- **system** (*ase.Atoms*) – System with positions and lattice.
- **groups** (*list[list[int]], optional*) – Atom index groups per site. If None, computed from *mol_id*.
- **rel_tol** (*float, optional*) – Relative tolerance for volume equality.
- **abs_tol** (*float, optional*) – Absolute tolerance for volume equality.
- **verbose** (*bool, optional*) – If True, print a report.

Returns
Summary with keys: *sum_voronoi_volume*, *cell_volume*, *abs_diff*, *rel_error*, *tolerances*, *ok*, *site_volumes*.

Return type
dict

C.5 Optimization

C.5.1 optimize_voronoi_overlap

molecular_modeling.optimize_voronoi_overlap.**optimize_voronoi_overlap**(*system: Atoms*, *groups: List[List[int]] | None = None*, *x0_euler_deg: ndarray | None = None*, ***, *select_sites: Sequence[int] | None = None*, *grid_num: Tuple[int, int, int] = (13, 13, 13)*, *angle_bounds: Tuple[float, float] = (-180.0, 180.0)*, *rng: int | None = None*, *log_dir: str | Path | None = None*) → Atoms

Optimize per-site Euler angles to maximize Voronoi IoU with uniform scaling.

Parameters

- **system** (*ase.Atoms*) – Input supercell with *mol_id* array and dictionary metadata in *system.info*.
- **groups** (*list[list[int]], optional*) – Atom index groups per site. If None, computed via *build_molecule_groups*.
- **x0_euler_deg** (*ndarray, optional*) – Initial Euler angles (N,3) in degrees. Defaults to zeros or *system.info["site_angles_deg"]*.
- **select_sites** (*sequence[int], optional*) – Subset of site indices to optimize. Others remain unchanged.

- **grid_num** (*tuple[int, int, int], optional*) – Grid counts for coarse search over (alpha, beta, gamma).
- **angle_bounds** (*tuple[float, float], optional*) – Inclusive bounds for grid search angles in degrees.
- **rng** (*int, optional*) – Random seed for Voronoi sampling; falls back to *system.info["rng"]*.
- **log_dir** (*str or Path, optional*) – If provided, writes per-site optimization logs to the directory.

Returns

The same *Atoms* with positions rotated/scaled in place and optimization results stored in *system.info* (*optimized_angles_deg*, *optimized_scales*, *optimized_iou*).

Return type

ase.Atoms

Notes

Per-site procedure:

1. Compute system Voronoi cell volume `V_sys` at the site COM.
2. Compute dictionary Voronoi cell volume `V_dict` (rotation-invariant).
3. Set uniform scale `s = (V_sys / V_dict)^(1/3)` (or 1 if `V_dict <= 0`).
4. Grid search then Nelder–Mead to maximize IoU using rotated+scaled dictionary seeds.
5. Apply rotation and uniform scaling about the site COM to atom coordinates.

C.6 Visualization

C.6.1 render_voronoi_overlays

molecular_modeling.render_voronoi_overlays.**render_voronoi_overlays**(*system: Atoms*, **, groups: List[List[int]] | None = None, angles_deg: ndarray | None = None, sites: Sequence[int] | None = None, out_dir: str | Path = './voronoi_pngs', dpi: int = 220, figsize: Tuple[float, float] = (6.0, 5.0), azim: float = -60.0, elev: float = 20.0, rng: int | None = None, show_overlap: bool = True*) → None

Render Voronoi overlays for selected sites and report volumes.

Parameters

- **system** (*ase.Atoms*) – System with *system.info* containing *site_mol_name*, *dict_meta*, and optional *optimized_angles_deg* / *optimized_scales*.
- **groups** (*list[list[int]], optional*) – Atom index groups per site. If None, computed with *build_molecule_groups*.
- **angles_deg** (*ndarray, optional*) – Euler angles (N,3) in degrees; defaults to optimized values or zeros.
- **sites** (*sequence[int], optional*) – Site indices to render; defaults to all.
- **out_dir** (*str or Path, optional*) – Output directory for PNGs.
- **dpi** (*int, optional*) – Figure DPI.
- **figsize** (*tuple[float, float], optional*) – Matplotlib figure size in inches.
- **azim** (*float, optional*) – Azimuth angle for 3D view.
- **elev** (*float, optional*) – Elevation angle for 3D view.
- **rng** (*int, optional*) – Random seed for Voronoi sampling; falls back to *system.info["rng"]*.
- **show_overlap** (*bool, optional*) – If True, plot overlap polyhedron in orange.

C.7 Output

C.7.1 write_structure

molecular_modeling.write_structure.**write_structure**(*system: Atoms*, **, outdir: str | Path = '.', verbose: bool = True*)

Write simulator-friendly structure files from an ASE *Atoms* object.

Parameters

- **system** (*ase.Atoms*) – System to export.
- **outdir** (*str or Path, optional*) – Output directory. Created if missing.
- **verbose** (*bool, optional*) – If True, print a summary of written files.
- **Writes**
- **------**
- **input** (*– Quantum ESPRESSO*)
- **inputs** (*– VASP*)
- **CIF** (*–*)

C.8 GUI

C.8.1 controller

Controller for GUI-backend integration.

Provides high-level operations that bridge GUI dialogs with the molecular modeling backend.

class molecular_modeling.gui.controller.**MolecularController**(*atoms: Atoms | None = None*)

Bases: object

Controller for molecular modeling operations.

Bridges GUI dialogs with the molecular modeling backend operations.

Parameters
atoms (*ase.Atoms, optional*) – Initial atoms object to operate on.

set_atoms(*atoms: Atoms*) → None

Set the current atoms object.

get_atoms() → Atoms | None

Get the current atoms object.

add_molecule(*molecule_name: str*, *position: Tuple[float, float, float]*, *rotation: Tuple[float, float, float] = (0.0, 0.0, 0.0)*) → bool

Add a molecule to the current system.

Parameters

- **molecule_name** (*str*) – Name of the molecule to add.
- **position** (*tuple[float, float, float]*) – Center of mass position in Cartesian coordinates.
- **rotation** (*tuple[float, float, float], optional*) – Euler angles (XYZ, degrees) for rotation.

Returns
True if molecule was added successfully.

Return type
bool

substitute_molecule(*molecule_index: int*, *new_molecule_name: str*) → bool

Replace a molecule at a specific index with a different molecule.

Preserves the orientation of the original molecule using principal component analysis.

Parameters

- **molecule_index** (*int*) – Index of the molecule group to replace.
- **new_molecule_name** (*str*) – Name of the new molecule to insert.

Returns
True if substitution was successful.

Return type
bool

global_atom_substitution(*old_symbol: str*, *new_symbol: str*) → int

Replace all atoms of one element with another.

Parameters

- **old_symbol** (*str*) – Element symbol to replace.
- **new_symbol** (*str*) – New element symbol.

Returns
Number of atoms replaced.

Return type
int

substitute_atom(*atom_index: int*, *new_symbol: str*) → bool

Replace a single atom with a new element.

Parameters

- **atom_index** (*int*) – Index of the atom to replace.
- **new_symbol** (*str*) – New element symbol.

Returns
True if substitution was successful.

Return type
bool

substitute_molecule_atoms(*molecule_index: int*, *new_symbols: List[str]*) → Tuple[bool, str]

Replace all atoms in a molecule with new symbols.

Parameters

- **molecule_index** (*int*) – Index of the molecule group.
- **new_symbols** (*List[str]*) – New element symbols (can be compact notation like "C6H6").

Returns
(success, error_message). error_message is empty if success.

Return type
tuple[bool, str]

set_supercell_repeat(*repeats: List[int]*) → None

Set the supercell repeat factors.

get_supercell_repeat() → List[int]

Get the current supercell repeat factors.

set_use_fractional(*use_fractional: bool*) → None

Set whether to use fractional coordinates.

get_use_fractional() → bool

Get whether fractional coordinates are used.

get_molecule_groups() → List[Dict]

Get information about all molecule groups in the system.

Returns
List of dictionaries with molecule group information.

Return type
list[dict]

get_selected_molecule_index(*selected_atoms: List[int]*) → int | None

Get the molecule index for a set of selected atoms.

Parameters
selected_atoms (*list[int]*) – List of selected atom indices.

Returns
Molecule index if all selected atoms belong to the same molecule.

Return type
int or None

molecular_modeling.gui.controller.**get_controller**() → MolecularController

Get the global controller instance.

Returns
The global controller instance.

Return type
MolecularController

molecular_modeling.gui.controller.**reset_controller**() → MolecularController

Reset and return a fresh global controller instance.

Use this when starting a new GUI session to avoid stale state.

Returns
A fresh controller instance.

Return type
MolecularController

C.8.2 molecule_registry

Molecule registry for dynamic loading of molecule definitions from TOML files.

Scans the `data/*.toml` directory and provides access to molecule configurations.

class molecular_modeling.gui.molecule_registry.**MoleculeRegistry**(*data_dir: str | Path | None = None*)

Bases: `object`

Registry for molecule definitions loaded from TOML files.

Scans the data/ directory for .toml files and loads molecule configurations.

Parameters
data_dir (*str or Path, optional*) – Path to the directory containing molecule TOML files. Defaults to "data" relative to the package root.

molecules

Dictionary mapping molecule names to their configurations.

Type
dict[str, dict]

reload() → None

Reload all molecule definitions from disk.

get_molecule_names() → List[str]

Get list of available molecule names.

Returns
Sorted list of molecule names.

Return type
list[str]

get_molecule(*name: str*) → dict | None

Get molecule configuration by name.

Parameters
name (*str*) – Molecule name.

Returns
Molecule configuration, or None if not found.

Return type
dict or None

get_symbols(*name: str*) → List[str]

Get expanded atomic symbols for a molecule.

Parameters
name (*str*) – Molecule name.

Returns
List of atomic symbols (expanded from compact notation).

Return type
list[str]

get_positions(*name: str*) → ndarray

Get atomic positions for a molecule.

Parameters
name (*str*) – Molecule name.

Returns
Atomic positions as (N, 3) array.

Return type
numpy.ndarray

get_num_atoms(*name: str*) → int

Get number of atoms in a molecule.

Parameters
name (*str*) – Molecule name.

Returns
Number of atoms.

Return type
int

get_unique_elements(*name: str*) → List[str]

Get unique elements in a molecule.

Parameters
name (*str*) – Molecule name.

Returns
Sorted list of unique element symbols.

Return type
list[str]

molecular_modeling.gui.molecule_registry.**get_registry**() → MoleculeRegistry

Get the global molecule registry instance.

Returns
The global registry instance.

Return type
MoleculeRegistry